MCP + RAG для 1С

Локальный поиск по BSL-коду и метаданным:
гибрид BM25 + cosine через RRF и честная оценка качества

Sergey Matyukhin

github.com/Smaturus/1c-mcp-rag-search

MCP Python SDK · Chroma · multilingual-e5-small · BM25 · RRF · flashrank

Задача и что получилось

Проблема

- Выгрузка 1С — тысячи .bsl-модулей и отчёт на десятки мегабайт
- Обычный grep не отвечает на смысловые вопросы
- Готовые поисковые решения требуют развёртывания и внешних API
- LLM-агенту нужен инструмент с предсказуемым качеством

Решение

- Свой MCP-сервер (stdio): 8 инструментов, включая маршрутизацию запросов
- Полностью локальный контур: e5-small + Chroma, работает на CPU
- Гибридный поиск: cosine + глобальный BM25 через RRF
- Измеримость: gold-set, file-level метрики, разброс по прогонам

Проект в цифрах

124 253
чанка в индексе

233 515
термов в BM25

0.209
nDCG@10 у hybrid

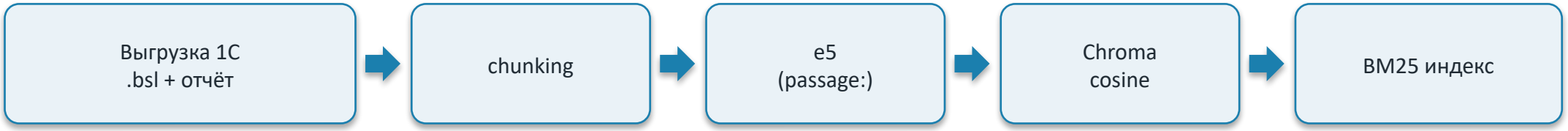
71
unit-тест, CI зелёный

Стек

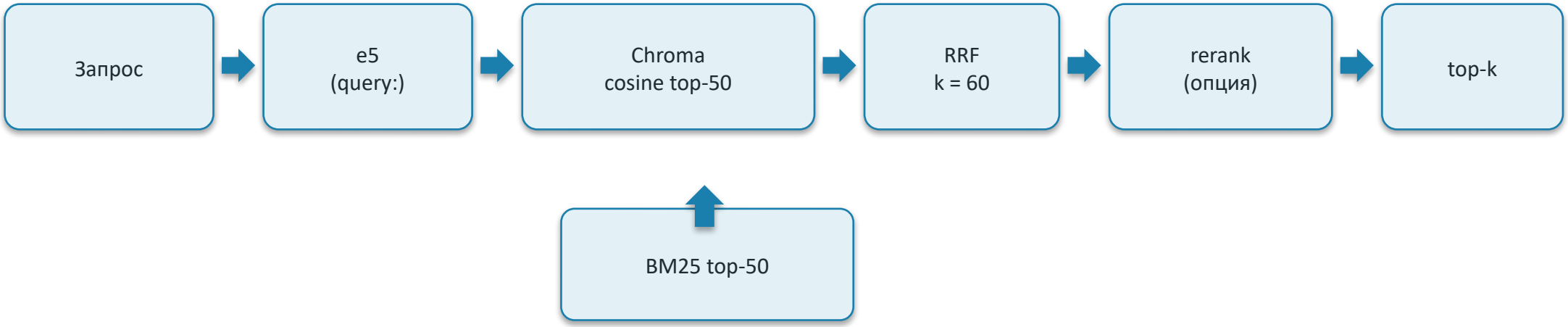
- Эмбеддинги: intfloat/multilingual-e5-small (384d, префиксы query: / passage:)
- Хранение: Chroma 1.5.9, HNSW cosine; лексика: собственный инвертированный индекс
- Гибрид: Reciprocal Rank Fusion (k = 60); reranker: flashrank + ms-marco-MultiBERT-L-12
- Интерфейс: MCP stdio (Cursor / ZCode), 4 инструмента; eval: 70 запросов, 4 конфигурации

Архитектура: два контура

Индексация (offline)



Поиск (online, MCP stdio)



Сверху — офлайн-подготовка индексов; ниже — онлайн-путь запроса. RRF объединяет два независимых ранжирования, reranker подключается опционально. Все данные и вычисления локальные: внешних API нет.

Индексы: векторы и лексика

Chroma (векторный)

- 124 253 чанка: code 100 445, metadata 23 808
- e5-small: 384 измерения, лимит 512 токенов на чанк
- Проверка: 32,1 % чанков были длиннее лимита — исправляем нарезкой по процедурам
- Префиксы e5 сохранены: passage: для документов, query: для запросов
- HNSW cosine; sync_threshold = 1e6 — защита от бага chroma#7090
- Размер базы 2.3 ГБ; старт пересобирает HNSW из WAL (~60–70 с)

BM25 (лексический)

- Инвертированный индекс: 233 515 термов, 6.1 млн постингов
- Сборка 9 с, 51 МБ на диске, читает документы прямо из sqlite
- Токенизация под 1С: кириллица, цифры, коды через дефис (НФ-001)
- Скоринг только по постингам запроса — без полного перебора корпуса
- Используется как второй источник ранжирования в RRF

MCP-сервер: инструменты для агента

Инструмент	Что делает
search_1c_metadata(query, limit)	grep-подобный BM25: точные имена, проводки, движения
search_knowledge_base(query, limit)	vector: «как сделать», «почему не работает», инструкции
search_hybrid(query, limit)	RRF: смешанные вопросы и требования — безопасный дефолт
route_query(query)	объясняет выбор: маршрут, причина, уверенность
search / codesearch / evaluate_retrieval	исторические инструменты и метрики по запросу

Подключение

- stdio-транспорт: сервер поднимается клиентом (Cursor mcp.json, ZCode mcp.servers)
- Маршрутизация — код с тестами, а не текст промпта: 24 кейса, 100 % точности
- Пример: route_query("проводки документа РеализацияТоваровУслуг") → search_1c_metadata

Гибридный поиск: Reciprocal Rank Fusion

$$score(d) = \Sigma\ 1 / (k + rank_i(d)),\quad k = 60$$

Почему RRF

- Cosine и BM25 дают скоры в несравнимых шкалах
- RRF использует только ранги — калибровка не нужна
- Точные имена ловит BM25, смысл — cosine, RRF складывает оба
- k = 60 сглаживает вклад первых позиций

Что дало (nDCG@10)

- vector: 0.176 ± 0.005
- hybrid (RRF): 0.209 ± 0.003 → +19 %
- Латентность: ~60 мс/запрос
- Разброс ±0.003 — вывод устойчив к пересборке HNSW

Пример. Запрос «ПередЗаписью»: BM25 поднимает модули с точным именем обработчика, cosine — смысловые совпадения. RRF складывает оба ранжирования без подбора весов.

Reranker: отрицательный результат, оставленный в отчёте

Как реализован

- flashrank 0.2.10, ONNX на CPU, без PyTorch
- ms-marco-MultiBERT-L-12, max_length = 128
- Вход: 20 кандидатов RRF; пассаж = путь файла + код
- Ленивая загрузка модели, кэш на диске

Результат

- nDCG@10: 0.209 → 0.069 (−67 %)
- Латентность: ×18 медленнее hybrid
- Гипотеза: модель обучена на веб-пассажах, а BSL — длинный повторяющийся код
- Решение: режим оставлен опцией, дефолт — hybrid (ADR 0002)
- Пайплайн не «крутился» под результат: проигравшая конфигурация осталась в таблице

Как измеряли качество

Данные и метрики

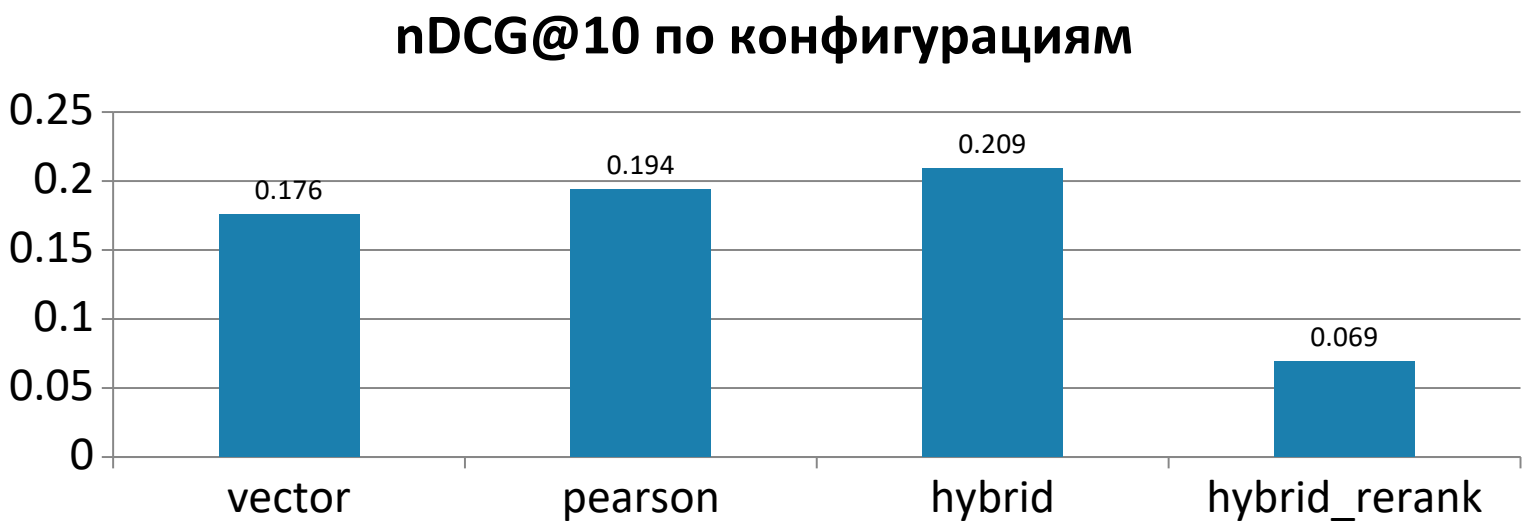
- gold-set v2: 70 запросов (30 курируемых + 40 по объектам конфигурации)
- Три типа: exact (точное имя), semantic (смысл), trap (ловушка)
- Релевантность — на уровне файла-источника, а не отдельного чанка
- Метрики: Hit@1, MRR@10, nDCG@10, Recall@10
- Каждый прогон — новый процесс: HNSW пересобирается, поэтому mean ± std

Почему так

- Чанковые эталоны давали ложно-нулевые метрики: правильный файл «не засчитывался»
- Файловая релевантность устойчива к смене chunk_size
- Recall@k нормирован на min(|relevant|, k)
- Разброс ±0.003 у hybrid — вывод воспроизводим, а не случайная удача

Результаты на gold-set v2 (70 запросов, k = 10)

Конфигурация	Hit@1	MRR@10	nDCG@10	Recall@10	мс/запрос
vector	0.076 ± 0.007	0.127 ± 0.005	0.176 ± 0.005	0.310 ± 0.015	~56
pearson	0.090 ± 0.007	0.139 ± 0.002	0.194 ± 0.006	0.348 ± 0.026	~99
hybrid (RRF)	0.090 ± 0.007	0.158 ± 0.003	0.209 ± 0.003	0.352 ± 0.007	~63
hybrid_rerank	0.014	0.047	0.069	0.136	~1100



Источник: data/eval/report_pipeline.md

hybrid — дефолтный режим:
лучший nDCG и MRR при минимальной латентности.

Pearson-гибрид — только baseline.

Решения и ограничения

Decision Log (ADR)

- 0001: RRF вместо взвешенного Pearson — выше метрики, без калибровки весов
- 0002: reranker оставлен опцией — ухудшает качество на BSL
- 0003: релевантность на уровне файла — устойчивые и воспроизводимые метрики
- 0006: маршрутизация запросов — код с тестами вместо текста промпта
- 0007: нарезка по процедурам — устранён лимит 512 токенов

Ограничения

- HNSW не персистится на диск: старт процесса ~60–70 с (защита от chroma#7090)
- Gold-set частично синтетический, абсолютные метрики скромные
- Один писатель индекса; параллельная индексация не поддерживается
- Reranker для кода — следующий контур (модель на коде или fine-tune)

Итоги и развитие

Что сделано

- MCP-сервер с 8 инструментами и локальным индексом на 124 тыс. чанков
- Гибридный retrieval: глобальный BM25 + RRF + опциональный cross-encoder
- Маршрутизация запросов: контракт с тестами, 100 % на размеченных кейсах
- Eval-контур: gold-set v2 (70 запросов), file-level метрики, mean ± std
- Инженерная обвязка: 71 тест, ruff, mypy, complexity-гейт, CI, ADR, CHANGELOG

Развитие

- Сравнить chunking по процедурам с текущим на gold-set v2; решить о смене дефолта
- Код-ориентированные эмбеддинги и новый baseline
- Reranker, предобученный на коде; ручная разметка 100+ запросов

github.com/Smaturus/1c-mcp-rag-search · smaturus@gmail.com